Departamento de Ingeniería Eléctrica
Facultad de Ingeniería
Universidad de Concepción
Concepción, Chile.

549253-1 - Taller de aplicación TIC 1

Mini proyecto 2

Profesor: Vincenzo Caro Fuentes

Integrantes: Ambar Facuse Carvajal

Maximiliano Pacheco Contreras

Paloma Vilches Fredes

Concepción, *1 de noviembre de 2026*

Índice

1. Item 1.1: Desarrollo de Minijuegos e Interfaces **2**
1.1. Minijuego 1: Among-oides . . . 2
1.1.1. Esquemático del circuito implementado . . . 2
1.1.2. Diagrama en bloques del código implementado . . . 3
1.1.3. Mockup de la interfaz gráfica . . . 4
1.2. Minijuego 2: Grita o muere... . . . 5
1.2.1. Esquemático del circuito implementado . . . 5
1.2.2. Diagrama en bloques del código implementado . . . 6
1.2.3. Mockup de la interfaz gráfica . . . 8

2. Item 1.2: Gestión de Red y Comunicación en Lobby **9**
2.1. Configuración y Credenciales . . . 9
2.2. Gestión de Red SSH/SFTP y Polling . . . 10
2.3. Lógica de Creación del Mensaje . . . 11
2.4. El Controlador . . . 12

3. Item 1.3: Control de Etapas y Flujo de Juego **13**
3.1. Sincronización de Etapas . . . 13
3.2. Asignación y Reporte de Minijuegos . . . 13
3.3. Gestión de Sabotajes (Ronda 2) . . . 13
3.4. Diagrama de Integración Global del Sistema . . . 14

1. Item 1.1: Desarrollo de Minijuegos e Interfaces

1.1. Minijuego 1: Among-oides

1.1.1. Esquemático del circuito implementado

El circuito implementado utiliza una Raspberry Pi 5 como unidad principal, en la que tiene conectada el sensor *KY-040 Rotary Encoder*, encargado del funcionamiento principal del minijuego *Among-oides*. Este está a cargo del desplazamiento del personaje, de derecha a izquierda. Si esquiva exitosamente los asteroides, suma puntos, de lo contrario descuenta puntos.

Este minijuego utiliza solo conexión del sensor *KY-040* directo al Raspberry Pi 5.

A continuación se presenta el esquemático del circuito implementado para la actividad correspondiente.

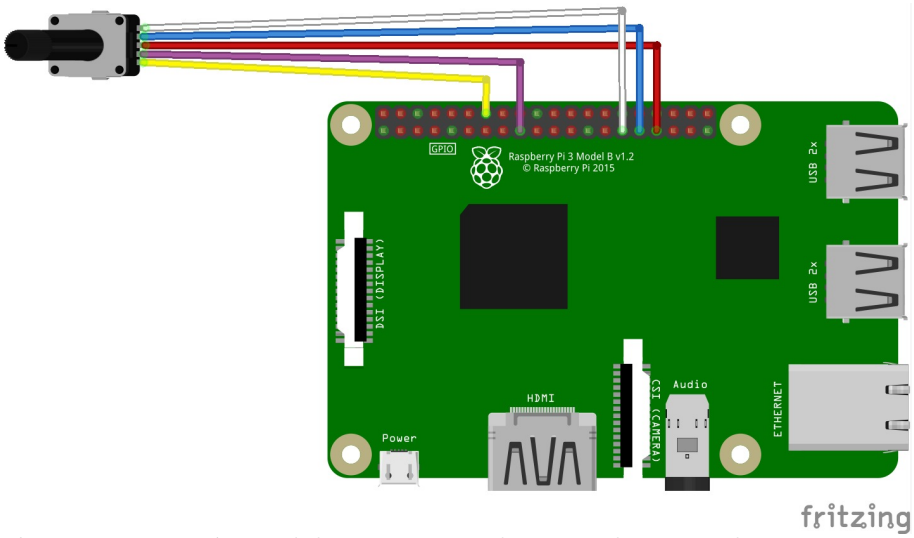


Fig. 1: Esquemático del circuito implementado para el Minijuego 1.

1.1.2. Diagrama en bloques del código implementado

En la siguiente figura se presenta el diagrama de bloques que representa la lógica general y el funcionamiento del código implementado en el minijuego 1 *Among-oides*.

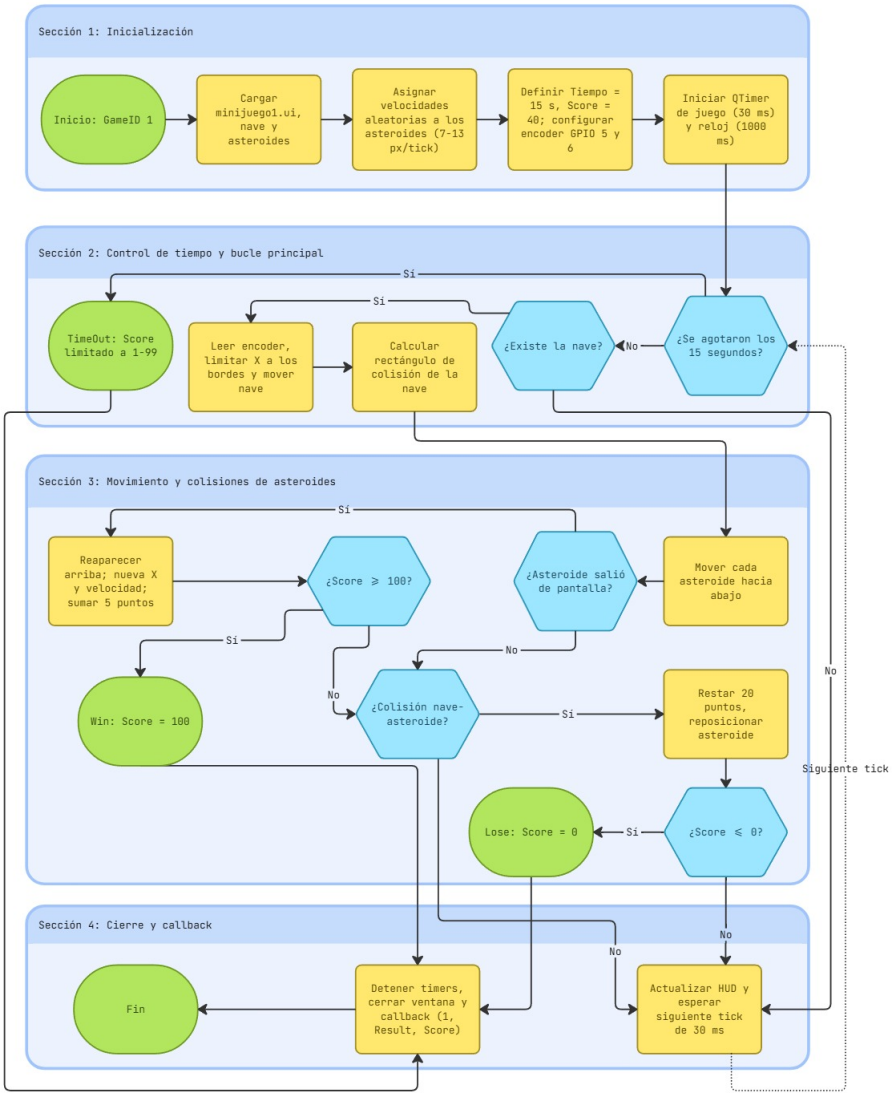
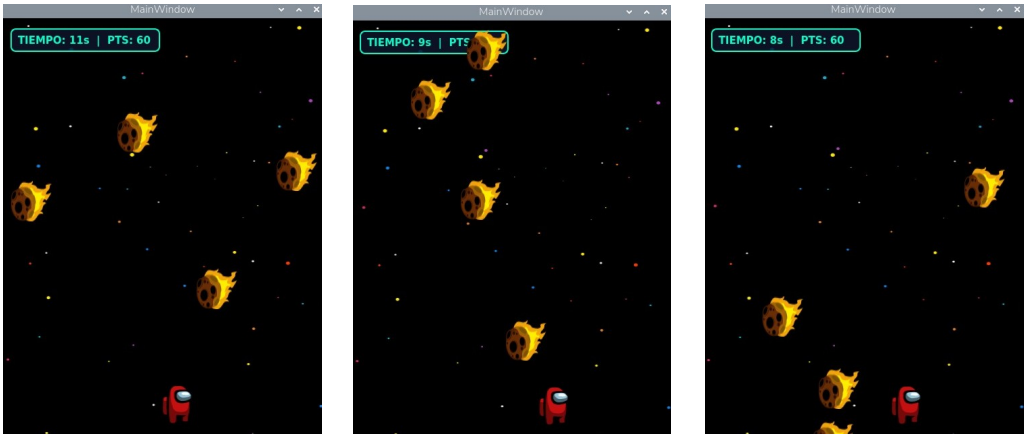


Fig. 2: Diagrama de flujo del Minijuego 1: *Among-oides*..

1.1.3. Mockup de la interfaz gráfica

A continuación, en la Figura 3, se exhiben las capturas de pantalla que ilustran el flujo y la experiencia visual de la interfaz gráfica desarrollada para el Minijuego 1 (*Among-oides*). La interfaz muestra en tiempo real la nave principal esquivando los asteroides, así como el panel superior con el tiempo transcurrido y la puntuación acumulada por el jugador.



(a) Inicio de partida (b) Desplazamiento (c) Esquive de obstáculos

Fig. 3: Interfaz gráfica del Minijuego 1: Esquivar asteroides en diferentes etapas del juego.

1.2. Minijuego 2: Grita o muere...

1.2.1. Esquemático del circuito implementado

El circuito implementado utiliza una Raspberry Pi 5 como unidad principal, junto a la respectiva Protoboard. Se utilizan dos sensores; el sensor *MAX9814 Electret Microphone Amplifier*, y el módulo *ADS-1115*, que funciona como conversor análogo-digital y permite recibir las señales sonoras del micrófono. En base a la amplitud del sonido, este permitirá a nuestro personaje del minijuego, saltar dependiendo la intensidad del sonido.

En este minijuego, los componentes se encuentran conectados a los pines GPIO de la Raspberry Pi, como también de la Protoboard.

A continuación se presenta el esquemático del circuito implementado para la actividad correspondiente.

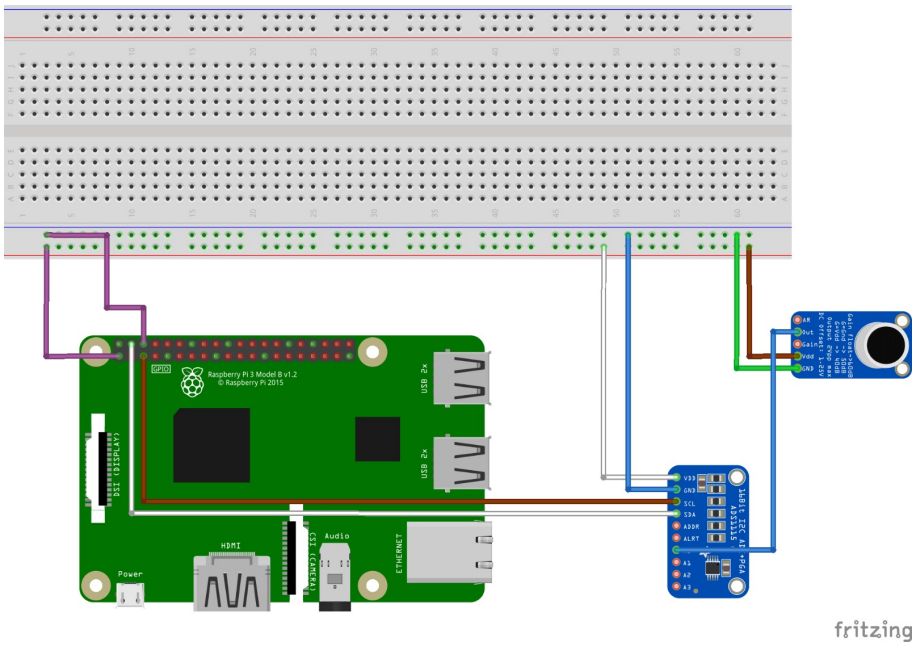


Fig. 4: Esquemático del circuito implementado para el Minijuego 2.

1.2.2. Diagrama en bloques del código implementado

En las siguientes figuras se detalla el diagrama de flujo dividido en dos partes para optimizar su legibilidad, describiendo la lógica de control, el procesamiento del micrófono y la física del juego.

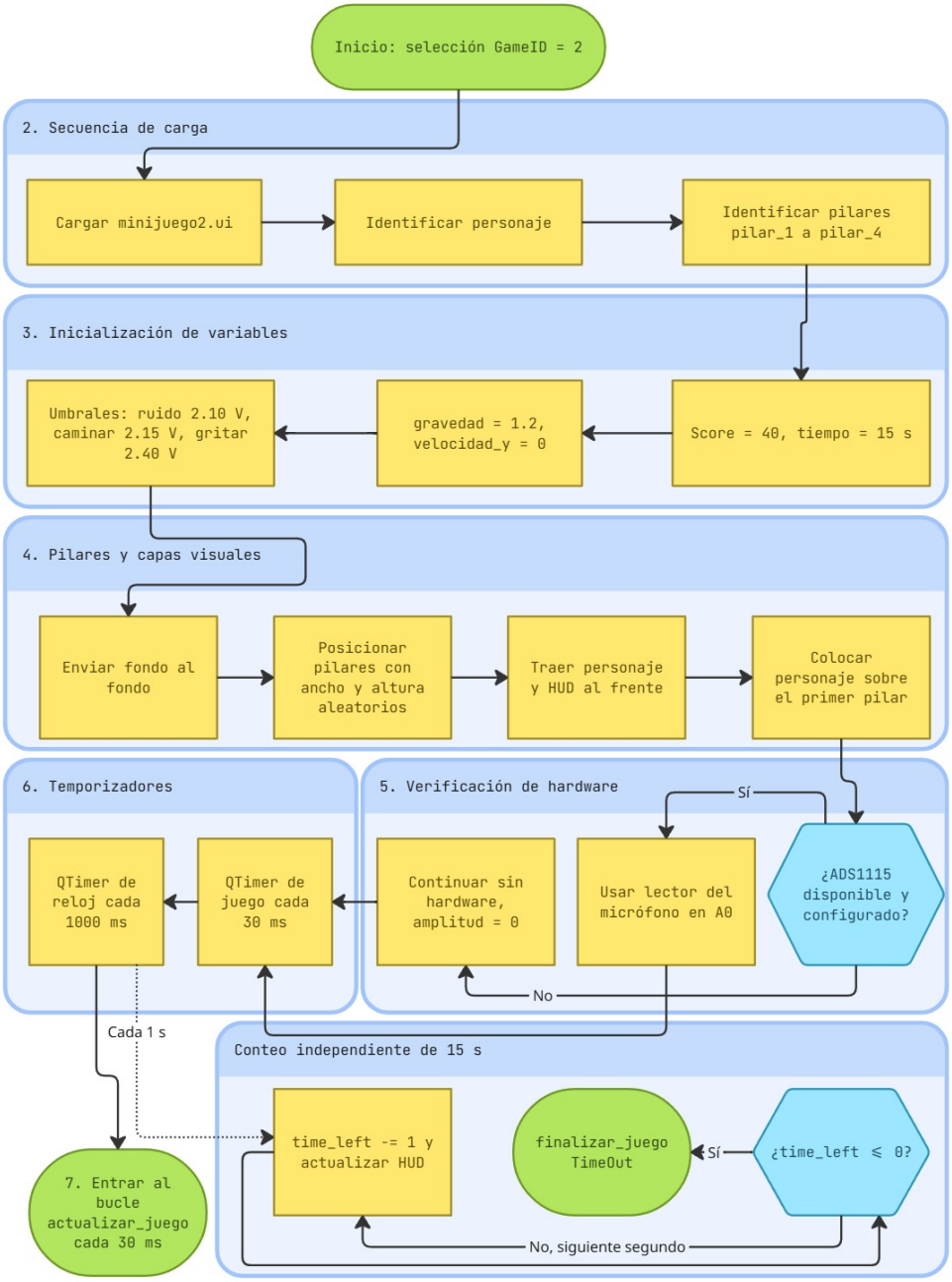


Fig. 5: Diagrama de flujo del Minijuego 2 (Parte 1: Inicialización, temporizadores y lectura de micrófono).

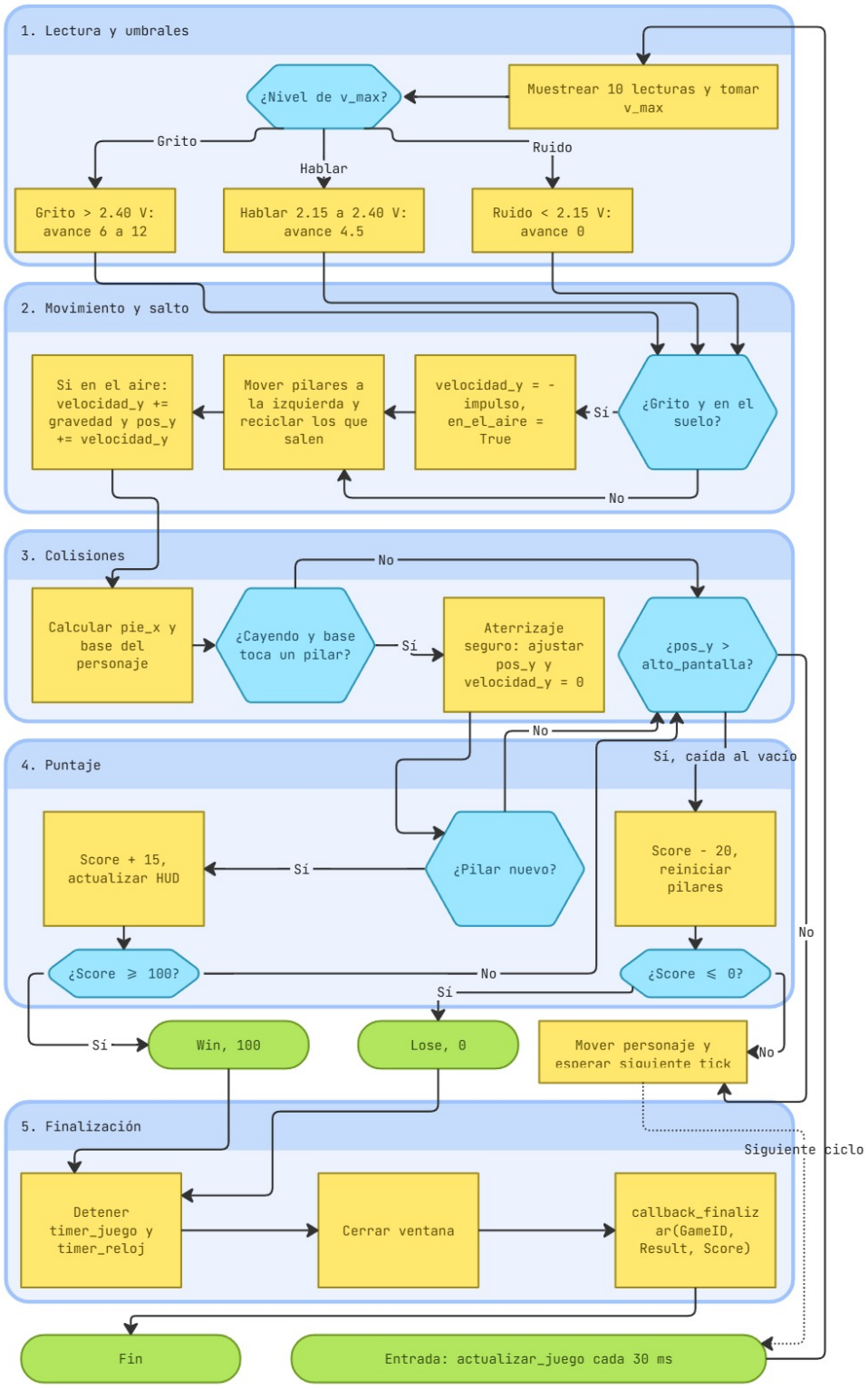
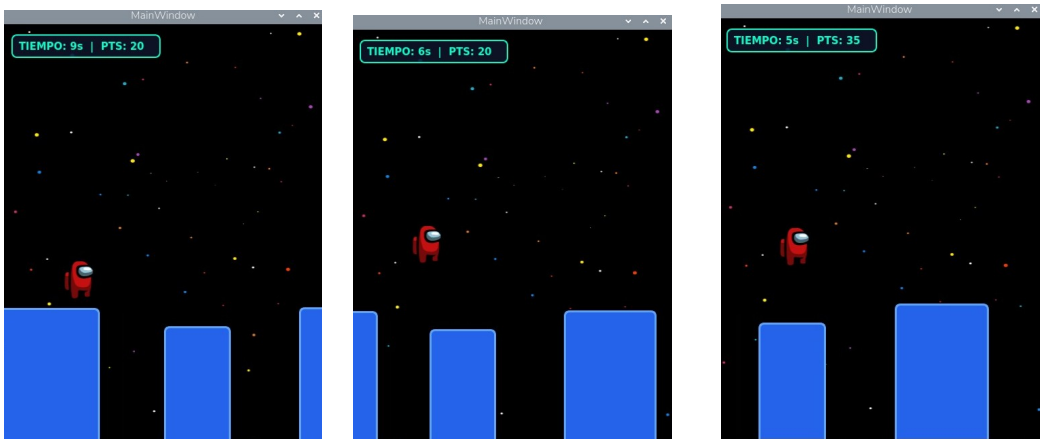


Fig. 6: Diagrama de flujo del Minijuego 2 (Parte 2: Física de saltos, colisiones, pilares y condiciones de término).

1.2.3. Mockup de la interfaz gráfica

En la Figura 7 se presentan las capturas de pantalla correspondientes a la interfaz gráfica del Minijuego 2 (*Grita o muere...*). Esta interfaz despliega en pantalla el escenario espacial en el que el personaje realiza saltos para esquivar los obstáculos del terreno. La altura y dinámica del salto son controladas en tiempo real mediante la amplitud de voz capturada por el micrófono MAX9814 y procesada con el conversor análogo-digital ADS-1115. Asimismo, en la esquina superior izquierda se ubica el panel indicador con el tiempo transcurrido y la puntuación (PTS) acumulada por el jugador.



(a) Posición inicial (b) Salto / Elevación (c) Avance sobre obstáculos

Fig. 7: Interfaz gráfica del Minijuego 2: *Grita o muere...* en distintas etapas del juego.

2. Item 1.2: Gestión de Red y Comunicación en Lobby

2.1. Configuración y Credenciales

Para establecer la comunicación con el host central sin hardcodear credenciales en el código fuente, el sistema utiliza el archivo de configuración `config_grupo.json`. Este archivo almacena los parámetros críticos asignados al grupo, tales como el identificador del jugador (`player_id`), el nombre de usuario y contraseña para la autenticación SSH, la dirección del servidor (`host`), el puerto (`port`), y las rutas remotas correspondientes a los directorios `/inbox` y `/outbox`. En la Figura 8 se muestra cada uno de los parámetros asigandos a nuestro grupo.


```
{
  "player_id": "P01",
  "username": "grupo01",
  "password": "4wQClvR1fnUjKmgW",
  "host": "tic-host.local",
  "port": 22,
  "remote_inbox": "/inbox",
  "remote_outbox": "/outbox"
}
```


Fig. 8: Credenciales Grupo 01.

2.2. Gestión de Red SSH/SFTP y Polling

La gestión de red se encuentra centralizada en la clase *conexion_host*, implementada mediante la biblioteca Paramiko. El cliente abre una sesión SFTP segura y ejecuta un bucle de sondeo (polling) continuo que consulta el directorio remoto */outbox* en intervalos de 0.2 a 0.5 segundos. Para evitar procesar archivos duplicados, las lecturas se filtran mediante un conjunto local (mensajes_leidos). Asimismo, el envío de mensajes hacia el /inbox se realiza de forma atómica (subiendo un archivo temporal *.tmp* y renombrándolo a *.json* para garantizar que el servidor no lea archivos incompletos.

En la Figura 9 se detalla su arquitectura de funcionamiento, la cual abarca la apertura de la sesión SFTP, la supervisión de la conexión y los mecanismos de intercambio de archivos con el host central.

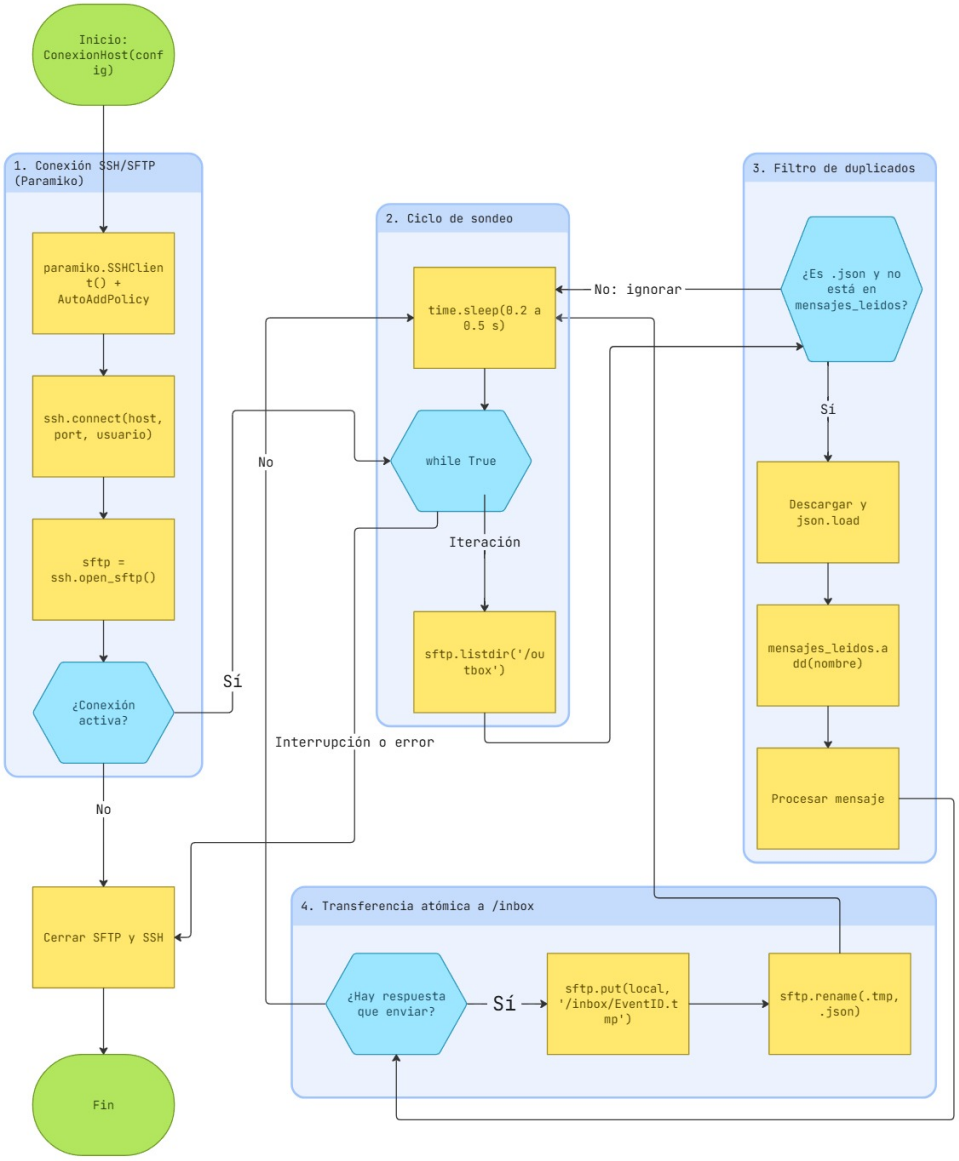


Fig. 9: Diagrama de flujo del módulo `ConexionHost` para la gestión SSH/SFTP, *polling* en `/outbox` y transferencia atómica en `/inbox`.

2.3. Lógica de Creación del Mensaje

Siguiendo las reglas del protocolo, la interacción inicial requiere estructurar diccionarios estrictos en formato *JSON*. El módulo *protocolo.py* encapsula la creación del evento *Join*, incorporando una marca de tiempo en formato ISO, el identificador único del mensaje (EventID) y la especificación de la etapa (Stage: "Lobby"). Esto permite que el host central reconozca e identifique unívocamente la solicitud de ingreso.

Para estructurar la comunicación con el servidor, el sistema implementa el módulo de protocolo. En la Figura 10 se ilustra el flujo lógico para la generación de mensajes estructurados y la validación de los acuses de recibo (ACK).

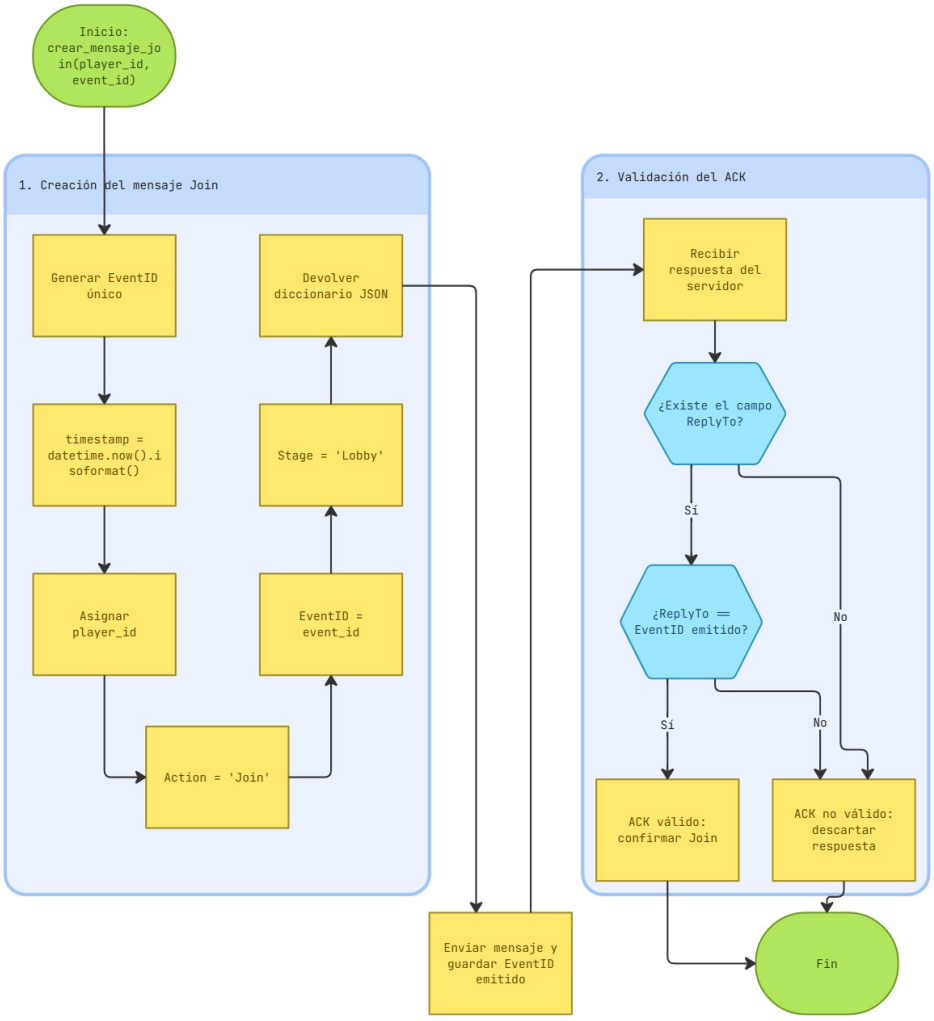


Fig. 10: Diagrama de flujo de la creación de mensajes y validación de ACK en el módulo de protocolo.

2.4. El Controlador

El archivo *main.py* actúa como el controlador principal que orquesta el flujo de ejecución. Tras inicializar la conexión con el host, transmite el evento *Join* y pone en marcha un bucle de control estricto que supervisa el directorio de salida. Cuando el sistema intercepta un mensaje con la acción *Accepted*, valida obligatoriamente que el campo *ReplyTo* coincida de manera estricta con el *EventID* original emitido, cerrando formalmente el ciclo de acuse de recibo (ACK) y dejando al cliente en espera activa dentro del lobby.

El archivo `main.py` orquesta la ejecución general del programa. La Figura 11 muestra el diagrama de flujo correspondiente al controlador principal, el cual gestiona la inicialización, el envío de la solicitud de ingreso y el bucle de espera activa en el Lobby.

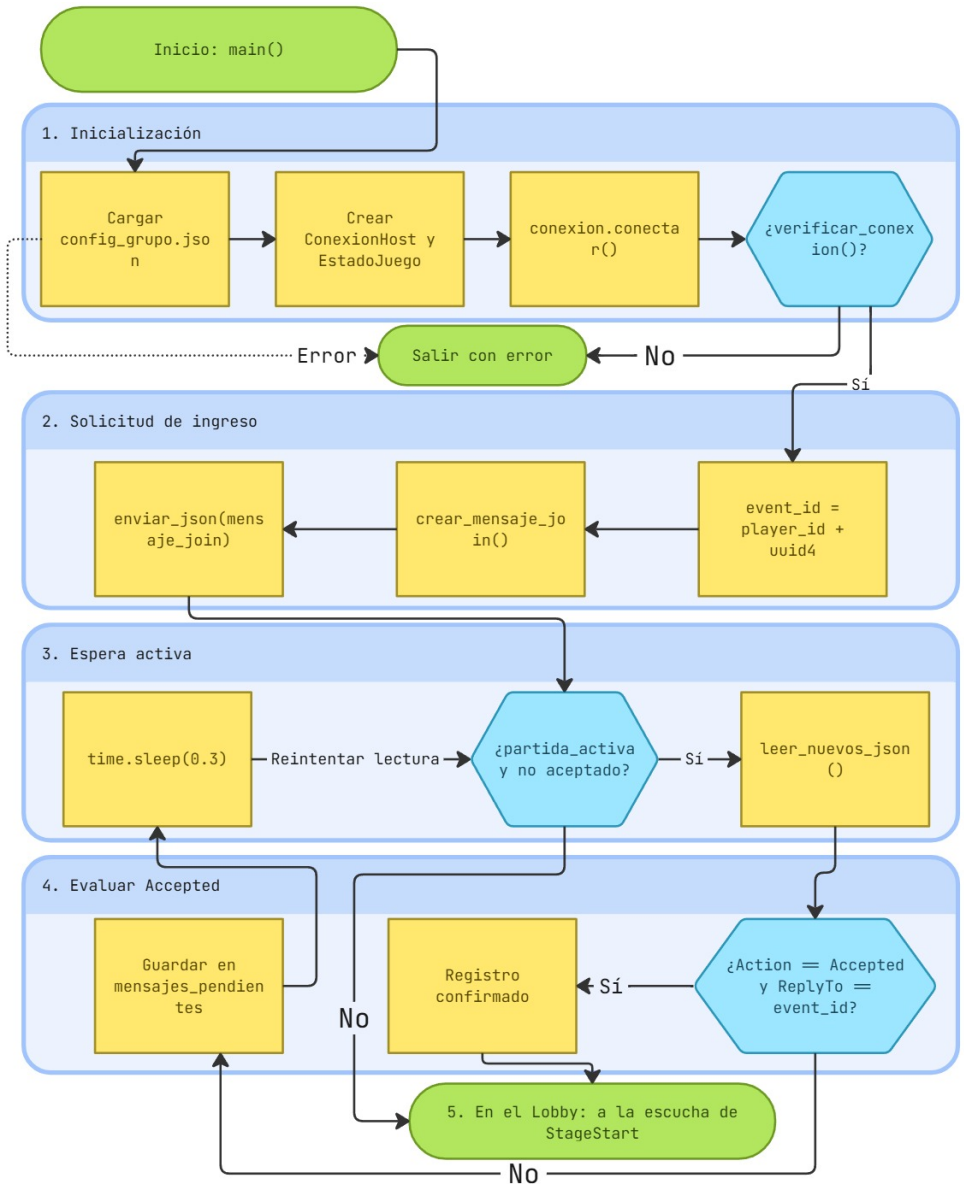


Fig. 11: Diagrama de flujo del controlador principal (`main.py`) para el ingreso y permanencia en el Lobby.

3. Item 1.3: Control de Etapas y Flujo de Juego

3.1. Sincronización de Etapas

Para coordinar el inicio de las competencias, el sistema procesa los mensajes del servidor mediante una máquina de estados. El cliente recibe la orden `StageStart`, responde confirmando su apresto con `StageReady` y aguarda la señal definitiva `StageActive` para habilitar la ronda.

3.2. Asignación y Reporte de Minijuegos

El controlador principal gestiona las instrucciones `Assign` enviadas por el servidor, desplegando la interfaz en PyQt6 correspondiente al *GameID*. Mientras el usuario juega, un temporizador secundario supervisa la red en segundo plano; al finalizar, el sistema reporta el puntaje mediante el evento `Result` y espera el acuse de recibo `ResultAccepted`.

3.3. Gestión de Sabotajes (Ronda 2)

Durante la segunda etapa, el cliente procesa la acción `Sabotage` para capturar efectos adversos enviados por el servidor (tales como retrasos, bloqueos o sustracción de puntos) y aplicarlos sobre la sesión del jugador.

3.4. Diagrama de Integración Global del Sistema

En la Figura 12 se presenta el diagrama de bloques general que resume el flujo completo del sistema, integrando la conexión, el Lobby, las rondas y los minijuegos como bloques funcionales compactos.

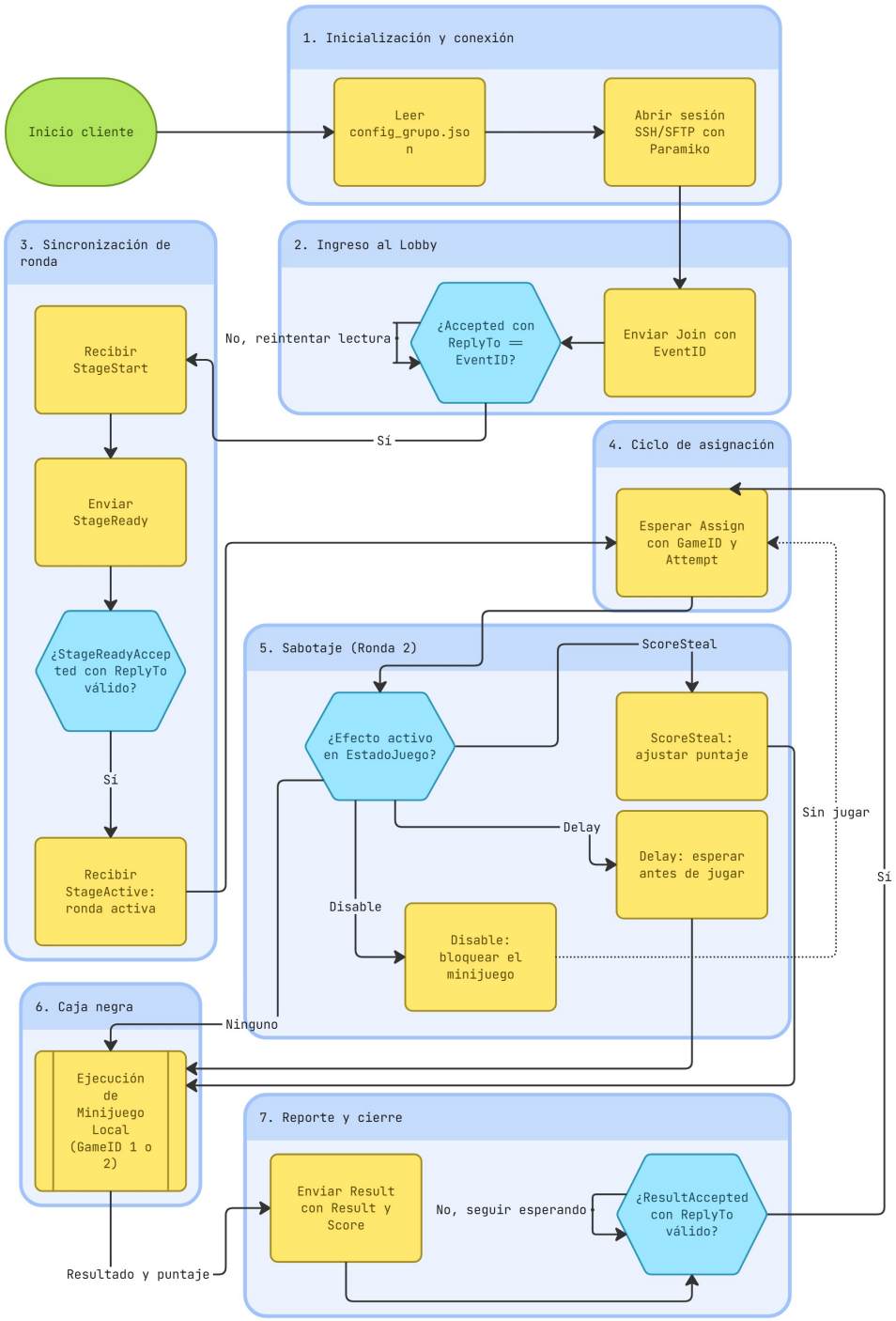


Fig. 12: Diagrama de flujo general de integración del cliente.